

cuGMEC Environment Setup and Build

This document describes how to install the dependencies required by cuGMEC, configure environment variables, and build the executable. The target environment is a Linux GPU workstation or GPU cluster.

1. Dependency Overview

Component	Requirement	Download
CUDA Toolkit	CUDA 12.x	https://developer.nvidia.com/cuda-toolkit
NCCL	Compatible with CUDA 12.x	https://developer.nvidia.com/nccl
cuDSS	0.8.0	https://docs.nvidia.com/cuda/cudss/
Intel MPI	Linux Offline Installer	https://www.intel.cn/content/www/cn/zh/developer/tools/oneapi/mpi-library-download.html

The example versions used in this document are CUDA 12.6, NCCL 2.23.4, cuDSS 0.8.0, and Intel MPI 2021.14.1.7. cuDSS must use version 0.8.0. CUDA, NCCL, the GPU driver, and Intel MPI must be mutually compatible.

2. Install CUDA and Set Environment Variables

Download the installer for your Linux distribution from the CUDA Toolkit website, and install it according to the official NVIDIA documentation. The following example uses CUDA 12.6:

```
# Edit the environment variable file
vim ~/.bashrc

# Add the following lines to ~/.bashrc. Replace the path with the actual CUDA directory.
export CUDA_HOME=/usr/local/cuda-12.6
export PATH=${CUDA_HOME}/bin${PATH:+:${PATH}}
export LD_LIBRARY_PATH=${CUDA_HOME}/lib64${LD_LIBRARY_PATH:+:${LD_LIBRARY_PATH}}

# Apply the environment variables and check the installation
source ~/.bashrc
nvcc --version
```

3. Install NCCL and Set Environment Variables

Download a local installer or archive package from the NCCL website that matches your CUDA version, and copy it to the target cluster. The following example uses NCCL 2.23.4 for CUDA 12.6:

```
# Extract and rename
tar xvf nccl_2.23.4-1+cuda12.6_x86_64.txz
mv nccl_2.23.4-1+cuda12.6_x86_64 nccl

# Edit the environment variable file
vim ~/.bashrc

# Add the following lines to ~/.bashrc. Replace the path with the actual NCCL directory.
export NCCL_HOME=/path/to/nccl
export LD_LIBRARY_PATH=${NCCL_HOME}/lib${LD_LIBRARY_PATH:+:${LD_LIBRARY_PATH}}

# Apply the environment variables
source ~/.bashrc
```

4. Install cuDSS and Set Environment Variables

Download a Linux archive package from the cuDSS website that matches CUDA 12.x, and copy it to the target cluster. The following example uses cuDSS 0.4.0.2:

```
# Extract and rename
tar xvf libcudss-linux-x86_64-0.4.0.2_cuda12-archive.tar.xz
mv libcudss-linux-x86_64-0.4.0.2_cuda12-archive cudss

# Edit the environment variable file
vim ~/.bashrc

# Add the following lines to ~/.bashrc. Replace the path with the actual cuDSS directory.
export CUDSS_HOME=/path/to/cudss
export LD_LIBRARY_PATH=${CUDSS_HOME}/lib:${LD_LIBRARY_PATH}

# Apply the environment variables
source ~/.bashrc
```

5. Install Intel MPI and Set Environment Variables

On the Intel MPI download page, select the Linux offline installer and copy the `.sh` installer to the target cluster. The following example uses Intel MPI 2021.14.1.7:

```
# Run the installer
sh intel-mpi-2021.14.1.7_offline.sh

# Edit the environment variable file
vim ~/.bashrc

# Add the following line to ~/.bashrc. Replace the path with the actual Intel oneAPI directory.
source /path/to/intel/oneapi/setvars.sh

# Apply the environment variables and check the installation
source ~/.bashrc
mpirun --version
```

6. Build

The current `Makefile` uses the following variables:

Variable	Default	Description
<code>NVCC</code>	<code>nvcc</code>	CUDA compiler
<code>ARCH</code>	<code>sm_80</code>	Target GPU architecture
<code>NCCL_HOME</code>	Empty	NCCL installation directory
<code>CUDSS_HOME</code>	Empty	cuDSS installation directory

Before building, make sure the environment variables for CUDA, NCCL, cuDSS, and Intel MPI have been configured and applied. If `NCCL_HOME` and `CUDSS_HOME` have already been set in `~/.bashrc` as shown above, and `source ~/.bashrc` has been run, no paths need to be specified manually here. Run `make` directly.

```
make
```

After a successful build, the `cuGMEC` executable will be generated in the project root directory.

Common GPU architecture examples:

GPU	<code>ARCH</code>
NVIDIA A100	<code>sm_80</code>
NVIDIA A800	<code>sm_80</code>
NVIDIA RTX 4090	<code>sm_89</code>
NVIDIA H100	<code>sm_90</code>